SEMANTIC CLUSTERING · THE MEANING LAYER

FIVE NEWSROOMS. ONE STORY.

> An **embedding** is a vector: a fixed-length list of numbers that a language model computes for a piece of text. Texts with similar meanings usually produce vectors that lie close together, so the system can calculate how similar the texts are rather than rely on a reader's judgement.

When a president resigns, for example, five newsrooms report the event in five different ways. The articles may share no phrasing at all, and sometimes they are not even written in the same language. A program that groups them by common words will miss even a simple paraphrase.

The system represents each article as a point in a semantic space. It does not measure ordinary distance between the points. Instead, it compares the directions of their vectors using the cosine of the angle between them. The resulting score estimates whether the texts are related.

The topic group an article belongs to has its own row in the database and lasts longer than a day. A new report is compared with the centres of existing groups rather than with every other article. A PostgreSQL database with the **pgvector** extension performs that search.

768
NUMBERS PER ARTICLE
That is how many coordinates describe each text's position in semantic space

0.70
THRESHOLD TO JOIN A GROUP
Cosine similarity ranges from −1.00 to 1.00. An article joins a group when its similarity to that group's centre reaches 0.70; in the sample, the same group scores 0.9933 and a different one 0.4010

3
TEXTS PER CALL
A server may have 4 GB of memory and no swap, so texts are sent to the model in small batches

none
EXPIRY DATE
A result computed once does not change, so it never needs to be invalidated

PRIMER

MEANING EXPRESSED AS COORDINATES

Three properties make embeddings useful for this task.

01 A TEXT AS 768 COORDINATES

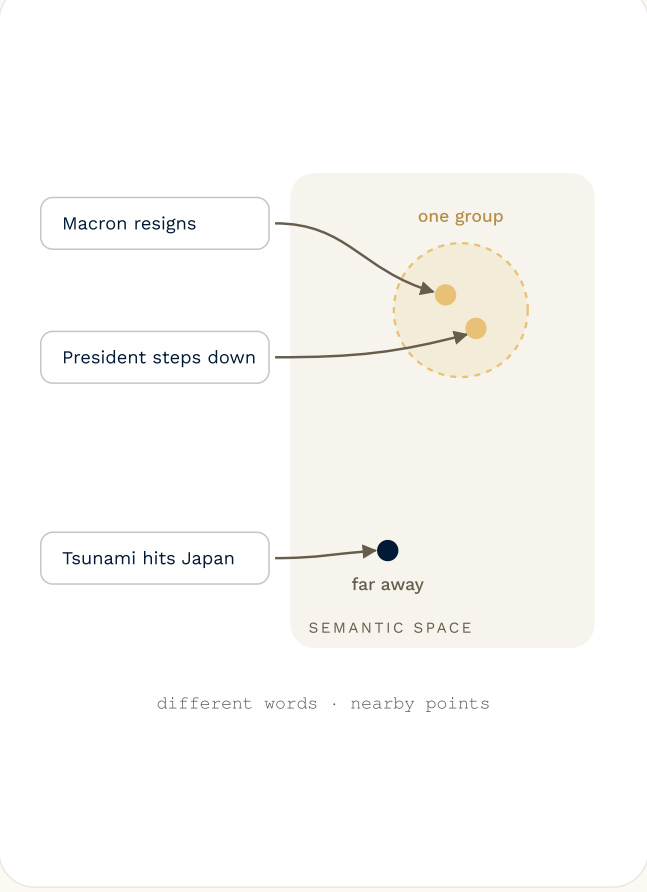
An embedding model turns a sentence into a list of 768 numbers. The result is neither a summary nor a set of keywords. These numbers are coordinates that place the text in a semantic space shared by every other text.

02 NEARBY VECTORS SUGGEST RELATED TEXTS

Texts about the same event often land in nearly the same place even when they use different words. **Cosine similarity** measures that closeness using the angle between their vectors; the card below, "How that score is computed", works through the whole calculation. This removes the need to write separate rules for synonyms and word order.

03 THE SAME TEXT ALWAYS GIVES THE SAME NUMBERS

Computing an embedding is deterministic: the same version of the model returns exactly the same vector for the same text. The result does not depend on when it is computed or on any text processed earlier. Storing the result avoids repeating the calculation.

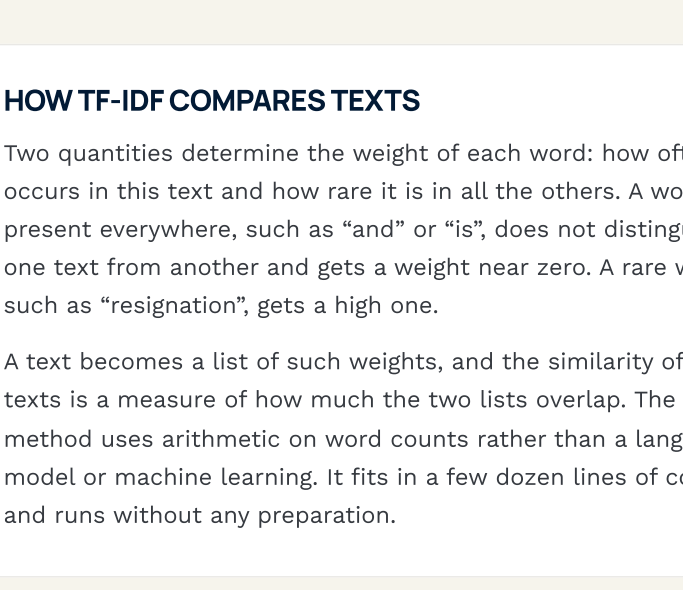


different words · nearby points

HOW THAT SCORE IS COMPUTED

The measure from point 02 is the **cosine of the angle** between two vectors — the same cosine similarity whose scale carries the 0.70 threshold. A vector is a list of numbers; this example uses two coordinates instead of 768 so the whole calculation fits in one diagram. A and B were chosen for the example; they are not model outputs. Both arrows start at [0, 0]: the first coordinate says how far to move the arrow's endpoint horizontally, the second how far to move it vertically. The angle itself is never needed — its cosine follows directly from the arrows' coordinates.

1 · READ THE COORDINATES



A = [1, 0]: 1 to the right and 0 upwards.
B = [0.8, 0.6]: 0.8 to the right and 0.6 upwards.

2 · CALCULATE DIRECTION SIMILARITY

$$\cos(\theta) = \frac{A \cdot B}{|A| \times |B|}$$

$A \cdot B$ is the dot product: multiply matching pairs, then add the results.

$|A|$ and $|B|$ are the lengths of the arrows. Both must be greater than zero because division by zero is undefined.

1. Calculate the numerator. Multiply numbers in matching positions, then add: $(1 \times 0.8) + (0 \times 0.6) = 0.8$
2. Calculate the vector lengths using the Pythagorean theorem: square each coordinate, add the results and take the square root: $|A| = \sqrt{1^2 + 0^2} = 1$; $|B| = \sqrt{0.8^2 + 0.6^2} = \sqrt{0.64 + 0.36} = 1$
3. Divide the numerator by the product of the lengths. This removes the effect of arrow size, leaving a comparison of direction: $0.8 \div (1 \times 1) = 0.80$

0.80 The score of 0.80 describes similarity of direction. It does not mean an 80% probability that the texts describe the same event. The "Topic groups" section uses this measure to match an article to a group.

−1 · opposite 0 · perpendicular 1 · same direction

The diagram shows an angle of about 37° because $\arccos(0.80) \approx 37°$. The system does not need the angle itself. It uses the score of 0.80 directly when comparing texts.

THE TECHNICAL SIDE

The model used here produces vectors with 768 coordinates. To calculate the dot product, the program multiplies 768 pairs of numbers and adds the results. Ollama's `/api/embed` endpoint returns vectors normalised to length 1 (L2 normalisation), with possible small deviations due to numerical precision. For vectors of length 1, the denominator is 1, so the cosine equals the dot product. The full formula also works for vectors of other, nonzero lengths.

A REFERENCE POINT

WHY COUNTING WORDS ISN'T ENOUGH

Before embeddings became common, texts were often compared using **TF-IDF**. The method first counts words, then gives each one a weight based on how often it appears in a given text and how rare it is across the collection. It considers two texts similar when they contain the same high-weight words.

HOW TF-IDF COMPARES TEXTS

Two quantities determine the weight of each word: how often it occurs in this text and how rare it is in all the others. A word present everywhere, such as "and" or "is", does not distinguish one text from another and gets a weight near zero. A rare word, such as "resignation", gets a high one.

A text becomes a list of such weights, and the similarity of two texts is a measure of how much the two lists overlap. The method uses arithmetic on word counts rather than a language model or machine learning. It fits in a few dozen lines of code and runs without any preparation.

WHERE IT BREAKS DOWN

Two texts may say the same thing in different words. "The president steps down" and "Macron resigns" share no meaningful word, so their similarity is exactly zero.

To match those two sentences at all, you have to add a list of synonyms, a way of reducing words to their base forms, and a list of names. Someone has to prepare these by hand and repeat the work for every language.

THE HARDEST CASE: SEVERAL LANGUAGES AT ONCE

The same event may be covered in English, French and Polish at the same time. If two articles in different languages share no words, a word-counting method assigns them a similarity of zero even when they describe the same event. Without additional rules, everything would first have to be translated into one language. That would require another model, whose errors would also affect the results.

Inflected languages make it worse: three grammatical forms of one Polish noun are three different words to a word counter. A multilingual model can compare texts without separate rules for every language. Because it learned many languages at once, it maps texts from all of them into one shared space. A Polish sentence can therefore land close to its English counterpart.

QUESTION	TF-IDF	EMBEDDINGS
What does it compare?	words in common	the meaning of the whole text
How does it know what matters?	from how rare words are in the collection at hand	from a model that has already read an enormous number of texts
Will it spot the same thing said differently?	no, not until someone adds a synonym dictionary	usually, because the model is trained to compare meaning
Can it handle two languages?	not without extra handling, such as translation	yes, provided the model is multilingual
What about a name, a ticker, a statute number?	better: an exact hit on a rare word carries the most weight	worse: similar names can land close together
Is the score always the same?	no, it changes with the collection used to calculate word rarity	yes, the same text and model always give the same result
What does it cost?	very little; it is computed locally, with no extra tooling	the model uses 1.3 GB and takes a few seconds; the result is then stored permanently
Can you explain the result?	yes, you can see which words determined the result	no; it consists of 768 numbers, none of which means anything on its own

Neither method is better in every case. Search engines usually combine them: word matching handles a statute number well, while embeddings capture the meaning of a question. Here the task is to decide whether the articles describe the same event, so their meaning matters most.

WHY ONLY ONE METHOD REMAINED

At first, a setting determined the grouping method: development environments without access to the model used TF-IDF, while production used embeddings. As a result, the same batch of articles formed different groups on a developer's machine and in production, while one similarity threshold had to serve two different distributions of scores. When article groups produce signals that may inform investment decisions, "I got a different result" stops being a minor detail.

Word counting had one advantage: it did not depend on the model. That advantage disappeared after embeddings for the test dataset had been computed once and stored in a fixture alongside the code. Before each test, those vectors are loaded into the cache. Tests therefore use the same embeddings and the same grouping method as production without having to call the model.

MODEL AND INFRASTRUCTURE

WHAT COMPUTES THESE NUMBERS, AND WHERE DOES IT RUN?

The model had to support multiple languages, fit on the same server as the database and application, and run on private infrastructure without sending text elsewhere.

NOMIC-EMBED-TEXT-V2-MOE

This open model takes text and returns 768 numbers. It does not hold conversations, summarise text or answer questions.

The **MoE** in the name stands for *mixture of experts*: the model contains several specialised subnetworks but activates only some of them for any given piece of text. It can therefore provide the capabilities of a larger model while using about as many resources as a smaller one.

For this task, multilingual support matters most: a French wire story and its English write-up can map to nearby points in the space, without translation or hand-written rules.

SPECIFICATIONS

SIZE AND SHAPE

OUTPUT	768 numbers · can be shortened with some loss of quality
LANGUAGES	multilingual · around a hundred languages
HOW MUCH IT READS	~512 tokens (~2 paragraphs) · any text beyond that is cut off
WEIGHTS	open · downloaded once, kept locally
IN MEMORY	~1.3 GB · only while a batch is computed
WHERE IT RUNS	Ollama on the same server · no external network calls

The model reads about 512 tokens, so anything beyond that is cut off. This system assumes that the article's subject will be clear from its headline and first paragraph. Important information that appears later will not affect the vector.

? WHAT IS OLLAMA?

Ollama is a program that runs language models on your own computer or server. You install it once and download a model with a single command. The model then runs at a local address, much like a database that receives requests and returns results.

Ollama takes a list of texts and returns a list of vectors. The application calls it over HTTP only when a new result has to be computed. No text leaves the server, and local calls are not billed individually.

$ REQUIREMENTS FOR RUNNING IT

A model this size takes ~1.3 GB of memory, and only while a batch is being computed. It fits next to the database and the application itself on an ordinary VPS costing a few euros a month. It requires neither a graphics card nor an account with a cloud provider.

A server may have 4 GB of memory and no swap. In that configuration, the model is unloaded from memory immediately after each batch, and texts are sent to it three at a time. Reloading takes a few seconds and is needed only when a batch contains new content.

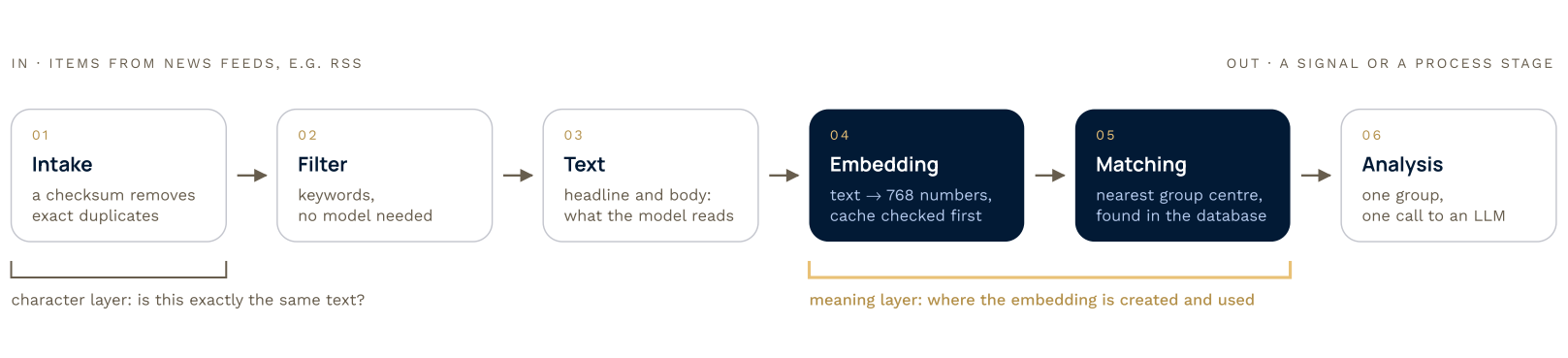
THE TECHNICAL SIDE

`POST /api/embed`, input in chunks of 3, `keep_alive: 0` on the last chunk (releasing memory). Address and model name come from configuration (`OLLAMA_EMBED_MODEL`).

THE ROUTE

WHERE EMBEDDINGS FIT INTO THE PROCESS

An article passes through six steps on its way from a news feed to a signal. Only two need an embedding. The others reduce the number of model calls and help ensure reliable results.


IN · ITEMS FROM NEWS FEEDS, E.G. RSS
OUT · A SIGNAL OR A PROCESS STAGE
01 Intake
a checksum removes exact duplicates
02 Filter
keywords, no model needed
03 Text
headline and body: what the model reads
04 Embedding
text → 768 numbers, cache checked first
05 Matching
nearest group centre, found in the database
06 Analysis
one group, one call to an LLM
character layer: is this exactly the same text?
meaning layer: where the embedding is created and used

Steps 01 to 03 do not analyse meaning: the first compares characters, the second looks for words from a list, and the third assembles the text for the model. Semantic analysis begins at step 04. Its result is stored as a row of numbers and used in step 05, where the database points to the group with the nearest centre. Step 06 receives a group with its new articles rather than a vector.

≡ INTAKE (STEP 01): HAS THIS TEXT BEEN HERE BEFORE?

The same wire copy may arrive from two feeds, or return after a dropped connection. A **checksum** detects it. This short string is computed from the entire text: the same text always produces the same checksum, while any change produces a different one. If the checksum is already in the database, the article is not stored again.

The check is binary and completely blind to meaning: moving a single comma produces a different checksum and a second row in the database.

≈ EMBEDDING AND GROUPING (STEPS 04 AND 05): IS THIS THE SAME STORY?

Five newsrooms describe one event in five different texts, so each checksum will be different. Embeddings let the program measure how close those texts are and group them even when they do not share a single sentence.

Each question needs a different tool. The program first compares content checksums, a fast operation that requires no model. Only texts that pass this stage get a vector and are compared with the centres of groups the database already holds. A Tuesday report can join a group opened on Monday, keeping the same group id.

01 ACCEPT IT, BUT ONLY ONCE

Articles sometimes arrive from news feeds more than once. A duplicate may result from a retry after a timeout or from the same wire copy appearing in two feeds. In both cases the content checksum is identical, so the program rejects the second copy.

THE TECHNICAL SIDE

`WorldEvent` · an identifier computed from the headline and body determines whether an event has already been seen. It uses a fast, non-cryptographic hash function (`xxh128(title + content)`). This is the **deduplication** key, deliberately a different one from the key the embedding cache uses.

IN · RSS feed items **OUT** · persisted events

02 REMOVE WHAT IS CLEARLY IRRELEVANT

A simple keyword filter comes next. It checks for required and disqualifying words, and optionally for trusted sources. It needs no model and reduces the number of texts passed to the stage that uses the model.

If no keywords are configured, nothing gets through. Passing every text in that case could hide the configuration mistake until the bill arrived.

The list still has to be prepared. A large language model can help during configuration, but it is not used during processing. The operator describes what they are looking for, and the model drafts a list of variants, related forms and equivalents in the source languages. The operator reviews and approves the final list.

THE TECHNICAL SIDE

`WorldEventFilter → WorldEventFilterSpec`: a CPU-only regex gate. When its configuration is empty, it rejects every text instead of allowing all of them through. Keyword suggestion is a separate path, called from the configuration screen: `KeywordSuggesterInterface::suggest(description, languages)`, adapter `AnthropicKeywordSuggester` behind a cache and a rate limiter.

IN · event snapshot **OUT** · array: WorldEventDescriptor objects

03 DECIDE WHAT THE MODEL WILL READ

For each article, the code joins the headline and body into one block. This exact text is sent to the model and determines both the resulting vector and its cache key.

THE TECHNICAL SIDE

`FilteredWorldEvent::getTextForClustering()` → `title . ' ' . content`.

OUT · array: texts

04 TURN THE TEXT INTO NUMBERS

This step uses more resources than the others, so it first checks whether the text has been processed before. The model receives only texts without a stored result.

IN · array: texts **OUT** · array: vectors of 768 floats

THE TECHNICAL SIDE

In PHP the result has type `array`: it contains vectors, each of which is an array of 768 `float` values. PHPDoc describes this shape more precisely as `list<list<float>>`.

05 JOIN A GROUP OR OPEN A NEW ONE

The program looks for the group most similar to the new article. If similarity is at least 0.70, the article joins that group; otherwise it opens a new one. The "Topic groups" section shows how the database finds a match and how the group's centre changes.

THE TECHNICAL SIDE

This describes matching to persistent topic groups, the clusters stored in the database. Multi-stage process tracking (`ProcessStepper`) builds separate clusters within a batch: it compares every pair and merges connected clusters, so links from A to B and B to C form cluster ABC.

IN · array: vectors of 768 floats **OUT** · event → group id

06 ONE GROUP, ONE ANALYSIS

A group is analysed once, not once per article, so five reports of the same event require one analysis in total. Grouping therefore also reduces the work at the final stage. The result is either a trading signal or an indication that a longer process has moved to its next stage.

THE TECHNICAL SIDE

`WorldEventCluster`: only groups that gained an article in this run go to analysis. `SignalGenerationKernel` (events → signals) receives events that already carry a group id and merely groups them by it (`AssignedClusterGrouping`). `ProcessStepper` groups within the batch, sorts clusters by size and stops at the first one to pass the stage gate.

OUT · Signal **OUT** · stage observation

STORING THE ANSWERS

HOW THE MODEL'S RESULTS REACH THE DATABASE

The cache stores each computed vector so the model does not have to compute it again. A single class manages the cache and sends the model texts that have no stored vector.

CHECK THE CACHE BEFORE COMPUTING

The program calculates a checksum for every text, as it does when catching duplicates, but bases this checksum on what the model actually sees. It then sends the checksums for the entire batch to the database in one query and asks which ones are already known.

Only the remaining texts go to the model. Each result is stored before it is used. When the same text appears again, the program reads its vector from the database instead of running the model a second time.

THE PATTERN: A DECORATOR

The class that queries the database cannot compute an embedding. The class that computes one knows nothing about the database. Both expose the same interface, so the database-aware class can wrap the model class. That is the decorator pattern.

The rest of the program asks for vectors without needing to know whether they came from the database or the model. The cache can therefore be removed, replaced or extended with another layer without changing the code that uses it.

THE DECORATOR IN CODE

This shortened version omits logging, the response-length check and the handling of texts repeated within one batch.

```
final readonly class CachedTextVectorizer implements TextVectorizerInterface
{
    public function __construct(
        private TextVectorizerInterface $inner,   // the model (Ollama)
        private EmbeddingStoreInterface $store,   // the embedding_cache table
        private string $modelId,
        private bool $ollamaEnabled,
    ) {}

    public function vectorize(array $texts): array
    {
        $hashOf  = array_map(self::keyFor(...), $texts);
        $vectors = $this->store->getMany($hashOf, $this->modelId);   // 1 query

        $misses = $this->collectMisses($texts, $hashOf, $vectors);

        if ([] !== $misses) {
            if (!$this->ollamaEnabled) {
                // a missing vector is an error; never silently fall back to another method
                throw new \RuntimeException('Embedding cache miss …');
            }

            $fresh = $this->inner->vectorize($misses);      // the only call to the model
            $this->store->putMany($fresh, $this->modelId);  // stored before it is used
            $vectors += $fresh;
        }

        return array_map(static fn (string $h) => $vectors[$h], $hashOf);
    }

    public static function keyFor(string $text): string
    {
        return hash('sha256', trim($text));   // keyed on the text the model reads
    }
}
```

The database key is the pair `(text checksum, model name)`, while the row contains the vector as `jsonb`. No expiry date is needed because the value does not change.

THREE POSSIBLE OUTCOMES

TEXT ALREADY KNOWN
any mode
When the same text appears again, the program reads the numbers from the database without running the model.

NEW TEXT, MODEL AVAILABLE
backend enabled
The model runs the computation locally, three texts at a time. The result is stored immediately, so the model will not be needed for that text again.

NEW TEXT, MODEL SWITCHED OFF
backend disabled
The program stops and reports which text has no result and which command can generate it. It does not fall back to a weaker method.

THE TECHNICAL SIDE

Table `embedding_cache`, composite key `(sha256(trim(text)), model_id)`, vector as `jsonb`, no TTL. Writes use `ON CONFLICT DO NOTHING`, so parallel workers don't collide. The model name is part of the key: once it changes, the same text no longer matches the old rows and the vector is computed afresh. The previous model's vectors stay in the table but are no longer read, so correctness does not depend on anyone remembering to clear the cache. Setting: `OLLAMA_ENABLED`. Cache initialisation: `bin/console app:embed:warm --days=1`.

TOPIC GROUPS

HOW AN ARTICLE JOINS A TOPIC GROUP

A topic group is a set of articles that the system has identified as reports about the same event. When another article arrives, the program must decide whether to add it to a known group or start a separate one. It uses embeddings, the numerical representations of those texts, to make that decision.

WHAT IS A GROUP'S CENTRE?

Each article has a vector of 768 numbers. To get one vector for the whole group, the program averages each position separately: first the first number from every article, then the second, and so on. The result is another list of 768 numbers. This is the group's centre.

The centre is a shared reference point for the group. The program compares the new article's vector with it. For a group with one article, the centre is that article's vector; with two, it is their average.

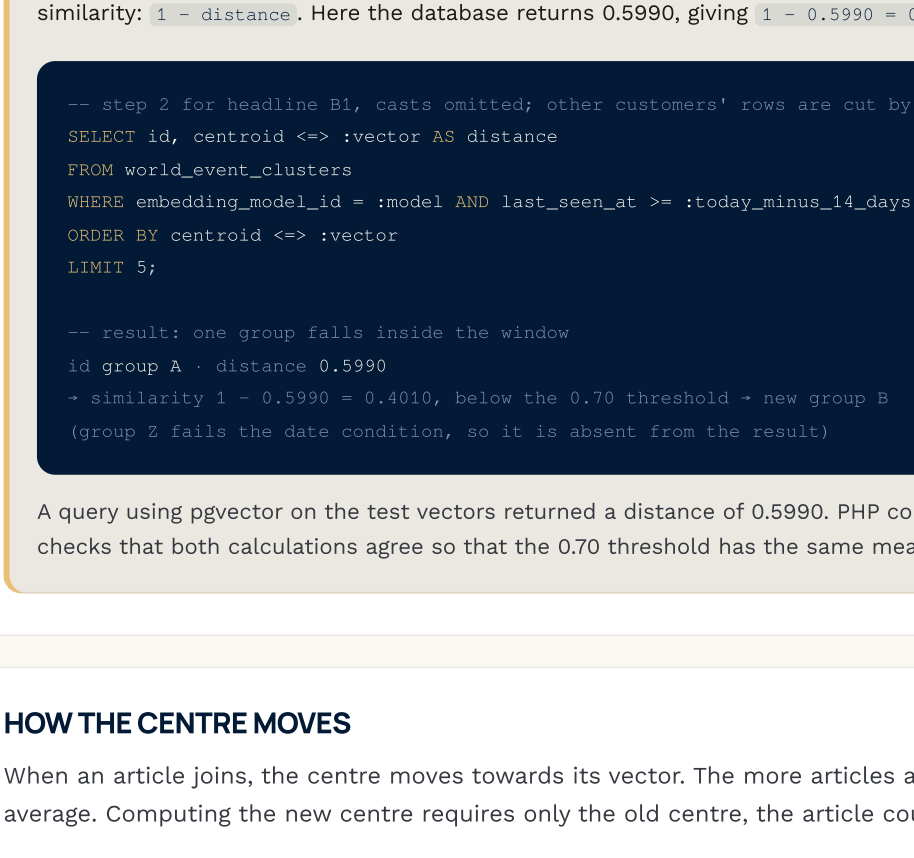
WHERE PGVECTOR HELPS

The model turns text into numbers. pgvector, a PostgreSQL extension, stores those numbers and compares vectors directly in the database. Here, the database compares the new article's vector with the stored group centres and returns the closest matches.

The program then checks whether the best match is similar enough. Finding the most similar group does not, by itself, mean the article belongs there.

DOES AN ELECTION REPORT BELONG IN THE RESIGNATION GROUP?

In this fictional example, the system has already grouped two headlines about Macron's resignation into group A. The diagram labels them A1 and A2. Now headline B1 arrives, about a snap election in France. It concerns the same country but a different event. The vector comparison will determine whether B1 joins group A.



The arrows show the sequence of actions. The positions of the boxes do not represent distances between vectors. The score of 0.4010 was computed from the same test vectors used in the "Specimen" section.

1. **TURN THE NEW TEXT INTO NUMBERS** The program retrieves the vector for headline B1 from the cache. If there is no stored result, it asks the model to compute one. `program`
2. **COMPARE IT WITH KNOWN GROUPS** The database compares B1's vector with the centres of the groups included in the search. In this example, only one group qualifies: A, about Macron's resignation. `database · pgvector`
3. **CHECK WHETHER THE SCORE IS HIGH ENOUGH** B1's similarity to the centre of A is **0.4010**. The scale runs from −1 to 1; a higher score means greater similarity. The program requires at least **0.70**, so B1 does not join A. `program`
4. **SAVE THE ARTICLE IN A SEPARATE GROUP** The program opens group B for the election report. It has only one article so far, so its centre is B1's vector. The article receives the new group's id. `program + database`

If another newsroom then sent headline B2 about the election, its similarity to group B would be **0.9942**. That exceeds the 0.70 threshold, so the program would add B2 to B. The group would have two articles, and its centre would be the average of their vectors. The next diagram shows how to calculate that average.

THE TECHNICAL SIDE

In the database this topic group is called a cluster: it searches `world_event_clusters`. It includes only this customer's groups with a report from the last 14 days and vectors from the same model. It computes the distance to every qualifying centre, without an approximate index, and returns up to five nearest matches. The program checks the best result. Group Z, whose last report was 16 days ago, does not qualify, so the query below returns only A.
Centres are stored in the `centroid` column of type `vector(768)`. The `<=>` operator returns cosine distance. The program converts it to similarity: `1 − distance`. Here the database returns 0.5990, giving `1 − 0.5990 = 0.4010`. The program's code defines the 0.70 threshold.

```
-- step 2 for headline B1, casts omitted; other customers' rows are cut by the RLS policy
SELECT id, centroid <=> :vector AS distance
FROM world_event_clusters
WHERE embedding_model_id = :model AND last_seen_at >= :today_minus_14_days
ORDER BY centroid <=> :vector
LIMIT 5;

-- result: one group falls inside the window
id group A · distance 0.5990
→ similarity 1 − 0.5990 = 0.4010, below the 0.70 threshold → new group B
(group Z fails the date condition, so it is absent from the result)
```

A query using pgvector on the test vectors returned a distance of 0.5990. PHP computes the same distance for this pair. An integration test checks that both calculations agree so that the 0.70 threshold has the same meaning in the database and the program.

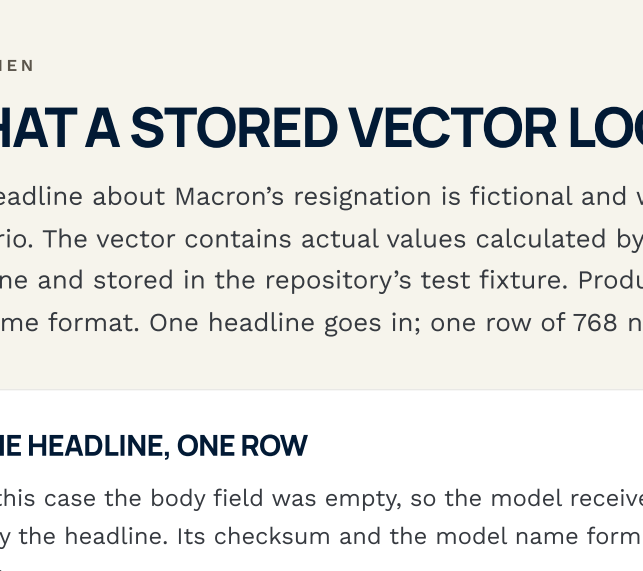
HOW THE CENTRE MOVES

When an article joins, the centre moves towards its vector. The more articles a group has, the less weight each new report carries in the average. Computing the new centre requires only the old centre, the article count and the new vector.

THE TECHNICAL SIDE

A two-coordinate example, using the same vectors as the "Primer" section. The group has one article, so its centre is that article's vector A = [1, 0]. Article B = [0.8, 0.6] arrives. Its similarity to the centre was calculated earlier: 0.80.

1 · OLD CENTRE, NEW ARTICLE, NEW CENTRE



The new centre is exactly halfway between the two vectors because both have a weight of ½. With ten articles, a new vector would move the centre only one eleventh of the way.

2 · CALCULATE THE NEW CENTRE

$$\text{centre}' = \frac{n \times \text{centre} + B}{n + 1}$$

1. Check the threshold. B scores 0.80 against the centre, which is at least 0.70, so B joins the group.
2. Substitute n = 1, because the group had one article: $(1 \times [1, 0] + [0.8, 0.6]) \div 2 = [1.8, 0.6] \div 2 = [0.9, 0.3]$
3. The count rises to 2 and the date of the latest report moves to today. The new centre has length $\sqrt{0.81 + 0.09} \approx 0.95$, less than 1. Cosine similarity still works because the formula divides by the vector lengths.

[0.9, 0.3] The new centre is the average of both vectors. The database updates it together with the article count in one statement, so concurrent joins do not overwrite each other's result.

In the group table the vectors have 768 coordinates, and the same `UPDATE` statement does this for all 768 at once.

HOW TOPIC GROUPS BEHAVE

ONE GROUP ACROSS MANY DAYS
a single identity
A Tuesday report joins the group opened on Monday. The program analyses that same group under its existing id; nothing new is created.

A SECOND RUN ON THE SAME DAY
no new articles
The program does not analyse a group again unless it has new articles.

A GROUP QUIET FOR 14 DAYS
outside the window
It returns as a new group. That is deliberate: a crisis in March and one in October produce similar vectors, yet for a market signal they are two situations with different opening prices.

THE TECHNICAL SIDE

The 0.70 threshold still needs validation for matching an article to a group centre. Two concurrent runs for one customer can open the same group twice and trigger an extra analysis.

SPECIMEN

WHAT A STORED VECTOR LOOKS LIKE

The headline about Macron's resignation is fictional and was created for the test scenario. The vector contains actual values calculated by the model for that headline and stored in the repository's test fixture. Production stores vectors in the same format. One headline goes in; one row of 768 numbers comes out.

ONE HEADLINE, ONE ROW

In this case the body field was empty, so the model received only the headline. Its checksum and the model name form the key.

WHAT THE MODEL READ	Macron resignation abdication power president france Macron resignation
TEXT CHECKSUM	7319e9beaf703b2351870019ae5446cf94cefe520f5f997f6326b3a357d62169
MODEL	nomic-embed-text-v2-moe
RESULT	768 numbers · length 1.00 · 5,745 bytes in the row

THE TECHNICAL SIDE

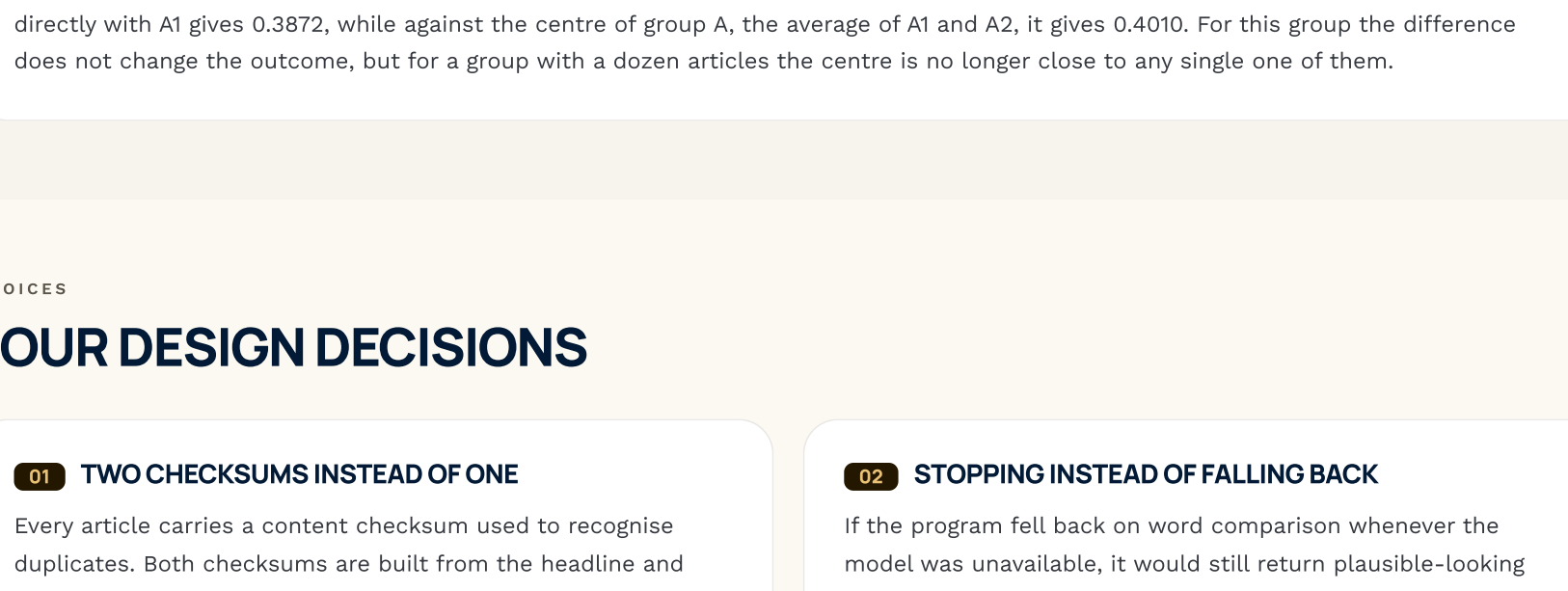
THE ROW ON DISK

The table has three columns, `VARCHAR(64)`, `VARCHAR(255)` and `JSONB`, plus a composite primary key. No additional index is needed because lookups use an exact key, never a similarity search. Similarity searches run against a different table, the one holding group centres, described in the "Topic groups" section.

```
-- SELECT * FROM embedding_cache LIMIT 1;
text_hash 7319e9beaf703b2351870019ae5446cf…
model_id nomic-embed-text-v2-moe
vector [0.021463, -0.003320, 0.013590, -0.022463,
… 763 more floats …
-0.026217, 0.054305]
```

THE FIRST 56 COORDINATES OF FOUR VECTORS

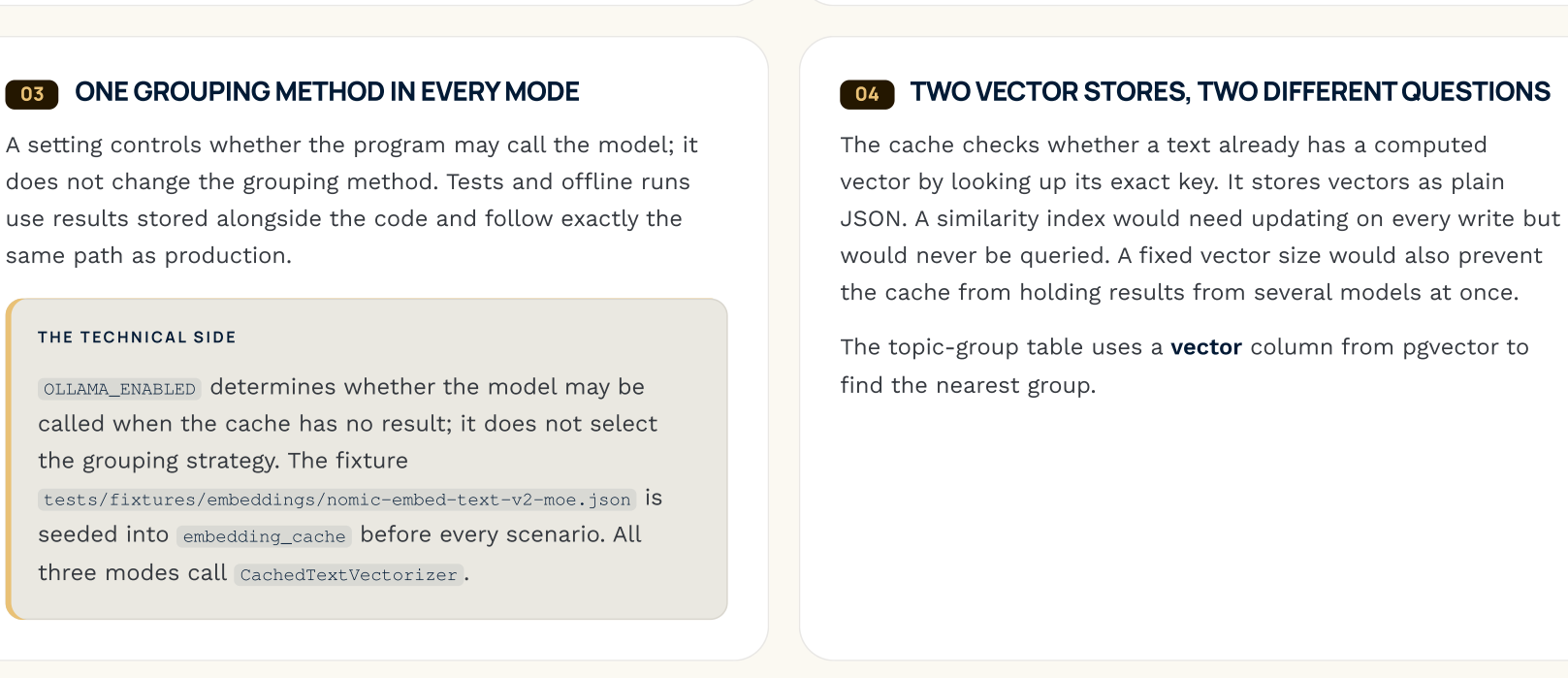
Each square represents one coordinate. Sand means a positive value and navy a negative one. The more intense the colour, the greater the absolute value. The first two headlines describe Macron's resignation using slightly different wording, yet their patterns line up square for square. The third also concerns French politics but describes a snap election, so its pattern is markedly different. The fourth, on an earthquake in Japan, resembles none of the others.



first 56 of 768 coordinates; cosine using this slice alone: 0.9955 / 0.4904 / 0.1429; this slice is only an illustration, and the system computes similarity from all 768 coordinates

WHERE THE THRESHOLD LANDS

The program compares three successive articles with the centre of group A. When the first arrives, group A contains only A1, so A1's vector is also its centre. By the time the next two arrive, group A contains two articles. The values come from a query against a database with the pgvector extension, run on the vectors in the test fixture.



A2 → centre of group A **0.9933**
Same resignation, second newsroom, reshuffled wording
↳ above the threshold: joins group A, and the centre moves halfway towards A2

B1 → centre of group A **0.4010**
Same country, same crisis, different event
↳ below the threshold: opens group B of its own

U1 → centre of group A **0.1292**
Japan earthquake, unrelated subject
↳ far below; the keyword filter would have dropped it first

Similarity to a centre is a different measurement from similarity to a single article. The strips above show the second kind: B1 compared directly with A1 gives 0.3872, while against the centre of group A, the average of A1 and A2, it gives 0.4010. For this group the difference does not change the outcome, but for a group with a dozen articles the centre is no longer close to any single one of them.

CHOICES

FOUR DESIGN DECISIONS

01 TWO CHECKSUMS INSTEAD OF ONE

Every article carries a content checksum used to recognise duplicates. Both checksums are built from the headline and body, but the content checksum cannot safely serve as the cache key. If the text sent to the model were assembled differently, the duplicate checksum would stay the same even though the model would read something else. The cache would then return numbers computed from a different text without warning. A second checksum takes only a few lines of code and prevents that mistake.

02 STOPPING INSTEAD OF FALLING BACK

If the program fell back on word comparison whenever the model was unavailable, it would still return plausible-looking groups. They would be arranged differently, however, and nothing in the output would reveal which method produced them. Here, article groups produce signals that may later inform a trading decision made outside the application. Incorrect grouping can therefore produce an incorrect signal. It is safer to stop with a clear message than to let a change in result quality go unnoticed.

03 ONE GROUPING METHOD IN EVERY MODE

A setting controls whether the program may call the model; it does not change the grouping method. Tests and offline runs use results stored alongside the code and follow exactly the same path as production.

THE TECHNICAL SIDE

`OLLAMA_ENABLED` determines whether the model may be called when the cache has no result; it does not select the grouping strategy. The fixture `tests/fixtures/embeddings/nomic-embed-text-v2-moe.json` is seeded into `embedding_cache` before every scenario. All three modes call `CachedTextVectorizer`.

04 TWO VECTOR STORES, TWO DIFFERENT QUESTIONS

The cache checks whether a text already has a computed vector by looking up its exact key. It stores vectors as plain JSON. A similarity index would need updating on every write but would never be queried. A fixed vector size would also prevent the cache from holding results from several models at once.

The topic-group table uses a **vector** column from pgvector to find the nearest group.

FURTHER APPLICATIONS

WHAT ELSE EMBEDDINGS CAN DO

Every use listed here relies on the same mechanism as grouping news reports. The model places content in a shared space, and the system compares the resulting vectors with a simple mathematical operation.

`what Proximtype does`
GROUPING AND TOPIC DISCOVERY
Texts without assigned categories form clusters, each representing one topic, including topics that nobody anticipated.

`what Proximtype does`
MATCHING ACROSS LANGUAGES
Reports in French, English and Polish can land in one group without prior translation.

`common`
SEMANTIC SEARCH
A model places queries and documents in the same space, so a search for "what can I do if my purchase is broken" can find the "returns policy for faulty goods", even though the two phrases share no words.

`common`
PROVIDING CONTEXT TO A MODEL (RAG)
Before the question reaches a large model, the system attaches passages from the organisation's own documents, selected for their similarity to the query. The model can then base its answer on that material instead of relying solely on its own memory.

`common`
DUPLICATES THAT AREN'T IDENTICAL
This may be a reposted listing with a changed phone number or the same case submitted a second time in different words. A checksum will treat the two as distinct, but an embedding can help link them.

`common`
CLASSIFICATION WITHOUT TRAINING
One sentence describes each category. The system assigns a text to the category with the nearest description. Adding a category requires one more sentence rather than another round of data collection and classifier training.

`common`
RECOMMENDATIONS AND "MORE LIKE THIS"
The system finds the nearest neighbours in the space and uses them to suggest similar articles, products, job ads or tickets without a hand-maintained taxonomy.

`common`
ROUTING SUPPORT TICKETS
A new ticket goes to the team whose past cases sit closest to it. The system uses that history rather than a set of rules.

`common`
SPOTTING THE ODD ONE OUT
A text lying far from everything the system has seen so far is either irrelevant noise or the first sign of something new. Either way, it is worth a look.

✕ WHAT AN EMBEDDING WILL NOT DO

It does not establish whether two texts are identical. The content checksum in step 01 does that. The similarity score is not an explanation either: it shows that two texts are close but not why. The model can use only patterns learned from its training data. For an event from last week, the embedding represents only what appears in the input text.

Port `MarketSignals/Domain/WorldEvent/TextVectorizerInterface` **Adapters** `CachedTextVectorizer → OllamaTextVectorizer`
Cache `embedding_cache (text_hash, model_id) → jsonb` **Topic groups** `world_event_clusters.centroid → vector(768), operator <=>`
Model `nomic-embed-text-v2-moe · 768 dimensions` **Cache initialisation** `bin/console app:embed:warm --days=1`